Project Interim Report

On

Online Course Management System

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

]

Submitted By

Shubham Kumar

ERP: RU-25-11719

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with Google

Under the Guidance of

Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The Online Course Management System is a console-based Java application designed to simulate a basic e-learning platform. It allows educational institutions or individual developers to manage courses, enroll students, and track their learning progress in an organised and efficient manner. The system is entirely built using Core Java and Object-Oriented Programming (OOP) concepts such as encapsulation, class interaction, and modular data handling through arrays and methods.

In today's digital age, online learning has become an essential component of education. However, managing multiple courses and student data manually is time-consuming and error-prone. This system addresses that problem by providing a structured, automated solution within a simple Java environment. It enables instructors to add courses, register students, enrol them in specific courses, update their progress, and display results — all from a command-line interface. The project demonstrates core programming skills and helps learners understand how real-world systems are built using OOP principles. It serves as an ideal foundation for a full-scale learning management system with future enhancements like a graphical interface and database connectivity.

Introduction

The Online Course Management System is a Java-based console application that provides a complete solution for managing an e-learning environment. It uses core OOP principles to model real-world entities such as courses and students, and enables interaction between these entities through well-defined methods. The system stores all data using arrays and demonstrates how object-oriented design can be applied to build functional software without any external libraries or frameworks.

Key Features of the Project:

1. Create and manage multiple courses with unique IDs, names, and durations.
2. Register students with unique IDs and names.
3. Enrol students into available courses.
4. Track and update each student's progress as a percentage.
5. Display a summary of all enrolled students and their progress for each course.
6. Demonstrate encapsulation through private fields with getter and setter methods.

Objectives:

- To design and implement a system that manages courses and students using Java classes and objects.
- To implement an enrolment system that links students to courses.
- To track student progress by updating and retrieving progress percentage for each course.
- To apply core OOP concepts including encapsulation, class interaction, and data handling.
- To build a working console-based application that can serve as a foundation for a full-scale Learning Management System (LMS).

Scope

This project is primarily suitable for small-scale educational environments such as coaching centres, training institutes, and college-level Java labs. It provides a basic but functional system for managing courses and student data without the need for a database or internet connection.

In the future, this system can be significantly extended and improved by integrating the following:

- A graphical user interface (GUI) using Java Swing or JavaFX.
- A relational database (MySQL/SQLite) to persist course and student data.
- A login and authentication system for students and instructors.
- Support for multiple course enrolments per student with progress tracking for each.

System Requirements

Hardware Requirements:

- Computer or laptop with a minimum of 4 GB RAM.
- Minimum 500 MB of free storage space for the Java Development Kit and project files.
- A keyboard and monitor for input and output operations.

Software Requirements:

Java Development Kit (JDK) – Version 8 or higher.

IDE: VS Code, Eclipse, IntelliJ IDEA, or EditPlus.

Windows, Linux, or macOS operating system.

Command Prompt or Terminal for running compiled Java programs.

Methodology

The system follows a step-by-step procedural approach to building the application. The methodology can be broken down as follows:

Step 1 – Creating the Course Class: A Course class is defined with private attributes such as courseName, courseId, and duration. Getter and setter methods are used to access and modify these fields, applying the principle of encapsulation. An array is used within the class to store the list of enrolled students.

Step 2 – Creating the Student Class: A Student class is defined with private attributes: studentName, studentId, progress, and a reference to the enrolled course. Methods like enroll(), updateProgress(), and getProgress() are defined to manage student-course interaction.

Step 3 – Enrolment Process: Student objects are linked to Course objects through the enroll() method. When a student enrols, the student reference is added to the course's enrolled student array, and the course reference is stored in the student object.

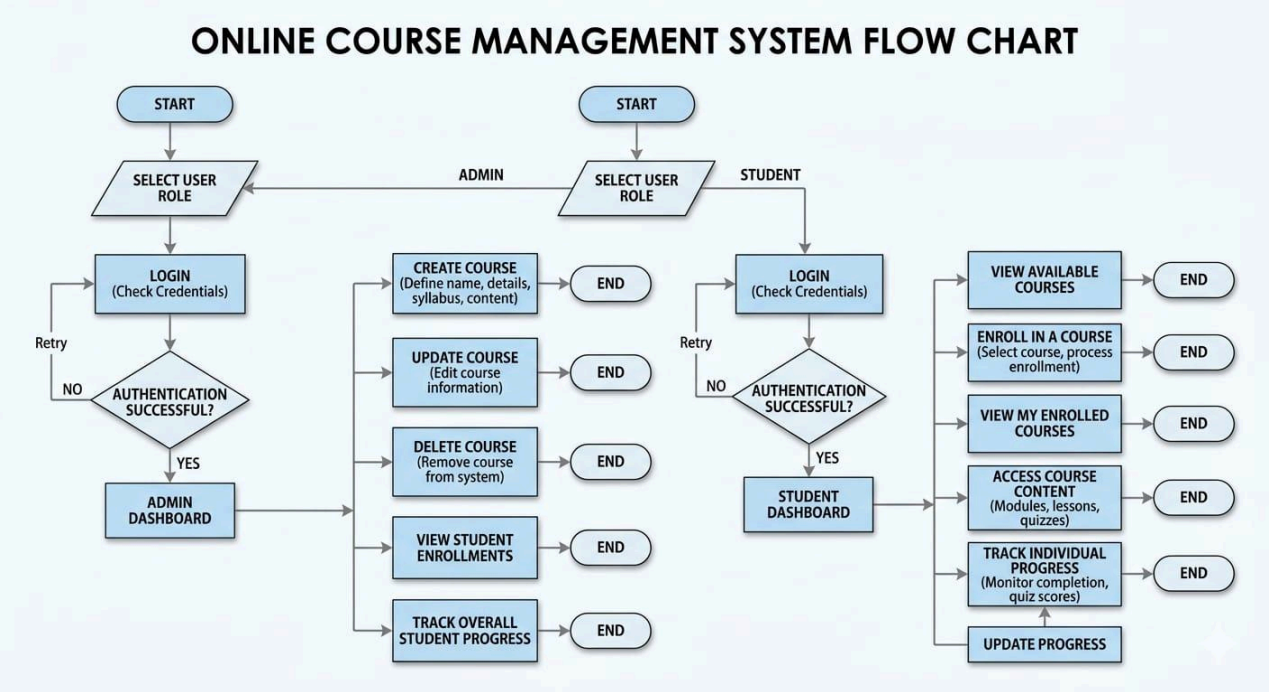
Step 4 – Progress Tracking: The updateProgress() method is called to update the progress field of a student object. The getProgress() method retrieves the current progress as an integer percentage (0–100).

Step 5 – Data Storage Using Arrays: Java arrays are used to store Course and Student objects in the main program. These arrays act as in-memory data stores that hold all the created objects during program execution.

Step 6 – Displaying Data: The main program iterates over the array of courses and, for each course, prints the course name, course ID, and a list of all enrolled students along with their progress percentages.

Flowchart

Program Flowchart:



Class Diagram Description:

Course Class

Attributes: courseName (String), courseId (String), duration (int), enrolledStudents (Student[])

Methods: getCourseId(), getCourseName(), getDuration(), addStudent()

Student Class

Attributes: studentName (String), studentId (String), progress (int), enrolledCourse (Course)

Methods: enroll(Course c), updateProgress(int percent), getProgress(), getStudentName()

Implementation

Course Class Logic: The Course class is the central entity of the system. It holds all the data related to a particular course. When a Course object is created, its name, ID, and duration are set through the constructor. An internal array stores references to enrolled Student objects. The class provides methods to retrieve course details and to add students.

Student Class Logic: The Student class represents a learner in the system. It stores the student's name, ID, and their progress value (as a percentage). It also holds a reference to the Course the student is enrolled in. The class uses encapsulation to keep all fields private and provides methods to enrol, update progress, and retrieve progress.

Enrolment Logic: When the enroll() method is called on a Student object with a Course object as a parameter, it sets the student's course reference to that course, and simultaneously adds the student to the course's enrolled students array. This creates a two-way relationship between the Student and Course objects.

Progress Tracking: Progress is stored as an integer (0 to 100) in each Student object. The updateProgress() method allows the progress to be set at any point in the program. The getProgress() method returns the current value, which is then used during the display phase.

Sample Output:

```
Course: Java Programming
Course ID: C101
Enrolled Students:
1. Rahul - Progress: 80%
2. Priya - Progress: 65%
```

Future Enhancements

The current system can be improved and extended in the following ways:

GUI using Java Swing: Replace the console interface with a graphical interface for easier interaction and better user experience.

Database Integration: Connect the system to a MySQL or SQLite database using JDBC to store and retrieve persistent data.

Search Functionality: Add the ability to search for a student or course by name or ID.

Multiple Course Enrolment: Allow a single student to enrol in multiple courses and track progress for each independently.

Menu-Driven System: Add a menu-driven console interface with options to add, view, update, and delete records interactively.

Login System: Implement a basic username-password login for students and instructors to access their respective dashboards.

Gantt Chart

A Gantt Chart is used to plan the project timeline by breaking the development process into tasks and assigning estimated durations to each. The following describes the timeline for the Online Course Management System project:

Week 1 – Planning: Define the problem statement, finalise the project scope, and identify required classes and data structures. Prepare the initial design document and class diagram.

Week 2 – Design: Design the class structure for Course and Student. Define all attributes and method signatures. Finalise the flowchart for the application logic.

Week 3 – Coding: Implement the Course class, Student class, and main program. Write the enrolment logic and progress tracking methods. Test individual methods as they are written.

Week 4 – Testing and Finalisation: Run the complete program with sample data. Verify correct enrolment, progress update, and display. Fix any errors and prepare the final project report.

Conclusion

The Online Course Management System has been successfully developed as a console-based Java application that demonstrates the practical application of Object-Oriented Programming concepts. The system effectively manages courses and students, supports enrolment, and tracks student progress in a

simple and structured manner.

This project reinforces the importance of OOP principles such as encapsulation, class interaction, and modularity in software development. By separating data and behaviour into distinct classes, the system is easy to understand, maintain, and extend. The use of arrays for data storage demonstrates how real-world problems can be modelled and solved even without complex data structures or external frameworks.

Overall, this project serves as a strong foundation for building a more advanced Learning Management System in the future, and it demonstrates the ability to design, implement, and test a complete Java application from scratch using core programming skills.

References

Books

[1] H. Schildt, *Java: The Complete Reference*, 12th ed. New York: McGraw-Hill Education, 2021.

[2] C. Horstmann, *Core Java Volume I – Fundamentals*, 12th ed. Upper Saddle River, NJ: Prentice Hall, 2021.

Websites

[3] Oracle, "Java Documentation," Oracle Corporation. [Online]. Available: https://docs.oracle.com/en/java/

[4] GeeksforGeeks, "Object Oriented Programming in Java," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/